

String pattern searching algorithm based on characters indices

1st Ivan Markić

Central Insurance Company Ltd
Mostar, Bosnia and Herzegovina
ivan.markic@outlook.com

2nd Maja Štula

Department of Electronics and
Computing
Faculty of Electrical Engineering,
Mechanical Engineering and Naval
Architecture in Split
Split, Croatia
kiki@fesb.hr

3rd Marija Zorić

Department of Electronics and
Computing
Faculty of Electrical Engineering,
Mechanical Engineering and Naval
Architecture in Split
Split, Croatia
marijuki@fesb.hr

In this article we shortly present new string pattern matching algorithm. The algorithm uses novel technique for skipping unnecessary comparisons in pattern searching phase. The pattern searching is applied in almost all branches of science such as bioinformatics, information security, text mining, etc. In the context of continuous increase of data, efficient algorithms are necessary to ensure that one can find a pattern in a sequence in a fast and accurate manner. Pattern searching solves the problem of finding a pattern exhibiting certain properties within a given sequence of symbols. Concept of the new algorithm presented in this article is based on a character index in a pattern, aiming at, but not limited to patterns in DNA sequences.

Keywords— string pattern matching, genome, DNA, biological data, exact string matching

I. INTRODUCTION

Comparison of sequences plays an important role in the theory of computer science because it represents a major challenge in a text processing. Most operating systems today use algorithms for string comparisons that usually run in a background for text processing tools. A string comparison consists of finding one or all occurrences of a string in a text.

Bioinformatics is a quantitative discipline of data analysis. Its main task is to perform computer analysis of biological macromolecules. Bioinformatics is often considered as a computational molecular biology. There are multiple views of what does bioinformatics mean. Bioinformatics is concerned with the development and use of computer tools in managing all types of biological data in one view, while the other view defines computer biology as theoretical development of algorithms used in bioinformatics. [1][2]

Most algorithms used in bioinformatics for analysis of biological sequences don't have ability and sophistication that truly reflect the reality. They often make wrong prediction that does not make sense when put in a biological context. The results of computations also depend on the available computing power, so many precise, but exhaustive algorithms, cannot be used due to the slow calculation. Instead, less accurate, but faster, algorithms with marked difference between accuracy and computation are being used. [3]

Next chapter contains short presentation of string pattern searching field with description of commonly used pattern

searching algorithms. Naïve, Boyer-Moore (BM) and Knuth-Morris-Pratt (KMP) algorithms are fundamental approaches so these algorithms are used for comparison with the proposed algorithm. The third chapter presents the new algorithm and comparison with naïve, BM and KMP algorithm on one example.

II. STRING PATTERN SEARCHING

A string that should be identified is called a pattern and is marked with the P sign, whereas the text is indicated by the T sign. The length of the pattern P is indicated by the lower case m, the length of the text T with lower case n, where $m, n > 0$ and $m \leq n$. Both strings consist of a series of a finite set of characters called the alphabet, marked with Σ (sigma) sign whose size is equal to the σ (small letter-sigma). For DNA, the finite set of characters is $\Sigma = \{A, C, G, T\}$ with size $\sigma = 4$. [1], [4]–[6]

An expression pattern matching is used to denote the process of matching strings and sequences. There are two main techniques for comparison of sequences. Exact string matching aims to find all occurrences, if any, of the exact pattern P in the text T and approximate string matching [7][8]. The problem of string matching consists of finding all the valid shifts that give the appearance of a pattern P in a given text T.

Position $s \in \{0, \dots, n-m\}$ in T is the correct shift if P appears with a shift s in T or, mathematically speaking equation (1):

$$\text{pattern}[i] = \text{text}[s+i], \forall i \in \{0, \dots, m-1\} \quad (1)$$

If s is a good shift, then equation (2):

$$P[0 \dots m-1] = T[s \dots s+m-1] \quad (2)$$

Otherwise, s is not a good shift.

An example of the matching problem for the word "GCAGAGAG" is shown in the Fig. 1. [9]

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
					G	C	A	G	A	G	A	G											

Fig. 1. An example of matching pattern in a DNA string

Comparison of sequences can be done in two ways, depending on whether the first observed sequence is a

pattern or the text. If we compare two characters, we usually say that they match if they are equal. Otherwise, they mismatch. The principle behind the work of algorithms for string comparison covers text scanning with the window of size m , commonly referred to as the sliding window mechanism. First, one must align the left end of the window and the text and then compare the characters from the window with the characters from the pattern. This activity is called the attempt. After an exact matching (or mismatch) of pattern with the text, the window is moved to the right. The same procedure is repeated over and over again until the right end of the window has reached the right end of the text. [1]

Almost all algorithms for the exact string matching can effectively skip every single characters' comparison, done in naïve algorithm, by finding out the structure of the pattern P or text T in normal time. Meanwhile, the rest of the string does not have to be known to the algorithm. Finding out a pattern P or text T structure is called the preprocessing phase. This phase is followed by the search phase whereby the information obtained in preprocessing is used to speed up the search phase of the pattern P in the text T . [1]

Boyer-Moore algorithm is often used in text editing programs, especially for search and substitute commands. The algorithm requires preprocessing phase constructing a table with data about skipping characters created for each character of the pattern. In the next step, the algorithm checks whether the pattern P is in the text T by making alignment of the pattern and the text. Then, comparison is made from right to left. Comparison is always made character by character. After the first iteration, the pattern P moves to the right and assumes the next index. Unlike classic search methods for pattern searches in the text (naïve), the Boyer-Moore algorithm uses three different techniques to speed up the process of searching a pattern within the text: search from right to left, bad character shift rule and good suffix shift rule. The bad character shift rule and good suffix shift rule are heuristic techniques applied in cases of mismatches between a pattern and a text and they shift the pattern P by the largest amount obtained by applying a specific rule. Fig. 2. shows how to combine bad character and good suffix rules. Recommendation is to take the maximum. The original Boyer-Moore algorithm uses a simple bad "character shift rule". The Boyer-Moore algorithm search phase is performed from right to left. In the first iteration of the comparison, if the characters at respective indices match, comparison is continued until character mismatch occurs. In case of a mismatch, the pattern P moves to the right of the T . Comparison is then carried out from the right end of the P . P is shifted by one place after each mismatch or after occurrence of P is detected. [1], [4], [6], [10]–[12]

Knuth-Morris-Pratt algorithm is used for the problem of exact matching executed in linear time. If we use a classic (naïve) algorithm for a given alignment of P with T , a certain number of first characters i of P match with their opposites in T . In case of a mismatch, in the following comparison, the classic algorithm should move P to the next index again, by making comparison from the left end

of the P . However, larger skips may occur in these cases. The algorithm requires the pattern preprocessing phase whose goal is to make a table with information about the next index of the pattern if mismatches occur. The basic working principle of the Knuth-Morris-Pratt algorithm is to make a bigger shift compared to classic algorithms. It is only necessary to be familiar with the location of mismatches in the P . [1][12][13]

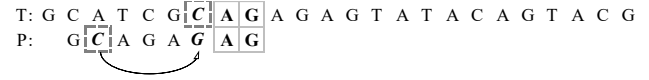


Fig. 2. Combining bad character and good suffix rules

III. ALGORITHM BASED ON CHARACTERS INDICES

After analysis of well-known algorithms for string matching a new concept is developed. This section presents a new concept of exact string pattern matching algorithm based on characters indices. This algorithm has the preprocessing stage where initial necessary information about characters indices in a pattern is gathered in a table with one row for each character from character set Σ (for the DNA sequences, $\sigma=4$, so the table has only four fields: A, C, T, G characters). In the pseudocode (Pseudocode 1.) this table is marked with M .

Characters comparisons are made by specific technique. This technique is going through pattern using even and odd positions in combination with information gathered from its preprocessing. Even positions are compared from left to right and odd position are compared from right to left. On this way searching process using available information from preprocessing on each skip if it is possible avoid unnecessary character comparison. Skipping as much as possible of character comparisons accelerates pattern searching and usually reduces the processing capacity thus making algorithms more efficient.

The proposed algorithm uses a temporary table that will serve for the shifting. This table is used and filled/updated in search phase which will in each shift contain the last index for each character that we found in a text in current shift of the pattern. In the pseudocode (Pseudocode 1.) this table is marked with N .

The searching begins comparing characters on even indices of a P with even indices in a T . If the characters on even indices match, comparison continues to the next even index and stores last matched index to the temporary table N . The comparison continues until the end is reached or the character mismatch.

When even characters mismatch, the pattern is shifted on the first character after mismatched character. When end is reached comparison moves on odd positioned characters. If all odd indices characters matched, then P is reported. After P is reported, P is shifted in the following way. Read next character from T and get second index from M . Shift P by a given value. If any odd indices mismatch, then mismatched character's index is checked versus first and second indices from M .

{Preprocessing stage}

Given the alphabet T (ACGT for DNA sequences)
 Given the pattern P
 Find first $F_P(i)$ and second $S_P(i)$ character indices in P
 Given the table M with alphabet indices values in P
 Given the table N with last character index in T for current P shift

{Search stage}

Loop through P and T
 Compare even and odd indices of $P(j)$ with $T(i)$
 Insert in N last character index in T for current P shift
 If $P(j)=T(i)$
 $N(\text{alphabet}) = \text{matched index of T}$
 Else if $P(j) \neq T(i)$
 search $F_P(i)$ and $S_P(i)$ of $T(i)$ in M
 check index $F_P(i)$ and $S_P(i)$ versus $T(i)$
 if $F_P(i)$ doesn't exist shift P
 if $F_P(i)$ and $S_P(i)$ after $T(i)$ shift P
 if $F_P(i)$ before and $S_P(i)$ after $T(i)$ shift P
 if $F_P(i)$ and $S_P(i)$ before $T(i)$ shift P
 if all even and odd pos of $P(j)$ and $T(i)$ are equal
 report an occ of P I T at index k
 if P if reported
 search $S_P(i)$ for $T(i+1)$
 if $S_P(i) > -1$ shift P

End

Pseudocode 1. Character indices based algorithm for exact pattern matching

Example shown in Fig. 3. explains new concept. Searching begins comparing character C with C on first even position of text and pattern (i.e. second position), they match in this case. Index for the position of character C from text in opposition to pattern is stored to the last current index table. The comparison continues on the next even position but odd third index since indices start from zero. Characters T and G are compared, and they mismatch. Index for position of character T from text in opposition to pattern is stored to last current index table. Position of first and second occurrence in pattern of mismatched character T is checked in preprocessing table. There are no occurrences of character T in pattern. After calculation of skipping it is possible to skip on next position after mismatched character T from text. Indexes in last current index table after skipping are reinitialized.

After skipping, comparison is made on fifth index. The characters C and G are compared, they mismatch. Position of character index G from text in opposition to pattern is stored to last current index table. The existence of mismatched character G from text is checked in preprocessing table. First position is found. It is also necessary to check if there are any position in last index table for additional skip. There are no positions for additional skip in this case. After calculation of skipping it is possible to skip by one position. Indexes in last current index table after skipping are reinitialized. The comparisons continue on sixth position of text. Comparing even and odd position the matching is found.

After matching is found, next character from text is taken and its existence is checked in preprocessing table. There are no occurrences for this character in preprocessing table. It is possible to skip pattern behind character. The comparison continues on fifteenth position of text until end

is reached. After the end of text is reached the algorithm is finished.

Preprocessing (table M):

	①	②
A	2	4
C	1	-1
G	0	3
T	-1	-1

Last current index:

A	-1	-1	-1	6,4,2	-1	5
C	-1	1	-1	1	-1	1
G	-1	-1	1	3,5,7	-1	3
T	-1	3	-1	-1	1	-1

Searching:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
#Comp	1	2	3	10	4	9	5	8	6	7					11	12	13	14						
#Shifts	G	C	A	T	C	G	C	A	G	A	G	A	G	T	A	T	A	C	A	G	T	A	C	G
1					G	C	A	G	A	G	A	G												
2					G	C	A	G	A	G	A	G												
3															G	C	A	G	A	G	A	G		
4																G	C	A	G	A	G	A	G	

Fig. 3. New algorithm example

IV. COMPARISON WITH OTHER ALGORITHMS

The two common algorithms for exact string pattern search, Boyer-Moore and Knuth-Morris-Pratt, have preprocessing stage while naïve search does not have preprocessing stage. It is included in comparison with the proposed new algorithm because it is simplest, basic exact pattern matching algorithm.

The new algorithm has $O(m)$ complexity in preprocessing phase and $O(m^2)$ complexity in search phase, Boyer-Moore has $O(m+\sigma)$ complexity in preprocessing phase and $O(mn)$ complexity in search phase, while Knuth-Morris-Pratt has $O(m)$ complexity in preprocessing phase and $O(n+m)$ complexity in search phase. Naïve search algorithm has only $O(m)$ complexity in search phase. [4]

For example shown in Fig. 3, the new algorithm has 4 shifts and 14 comparisons, Knuth-Morris-Pratt has 7 shifts and 18 comparisons, Boyer-Moore has 4 shifts and 17 comparisons and Naïve has 16 shifts and 30 comparisons. So, for this example of the pattern and the text, the new algorithm has better result than Knuth-Morris-Pratt and Naïve.

Algorithms should be compared on a large data sets, so further testing on large testing data sets is currently underway.

V. CONCLUSION AND FUTURE WORKS

In this paper we proposed and shortly described a novel algorithm for exact string pattern matching. We developed a new concept exploiting characters indices in a pattern being searched. The new algorithm consists of pattern preprocessing stage, during which information about characters indices in a pattern is gathered, and search stage. Also, information about matched character indices for each pattern shift is used to skip unnecessary character comparison. Skipping as much as possible characters comparison accelerates pattern search and usually reduces required processing capacity making algorithms more efficient.

Proposed approach is aimed primarily for biological, more precisely DNA sequences, but is not limited to them. The future work will include thorough testing on large text

sequences and representative data sets of a DNA pattern sequences that should yield better comparison with majority of existing algorithms uncovering possible advantages and disadvantages of the proposed string pattern searching algorithm based on characters indices.

VI. REFERENCES

- [1] D. Gusfield, "Algorithms on strings, trees, and sequences: computer science and computational biology," Theory and Practice, vol. 28, no. 4, p. 554, 1997.
- [2] P. Compeau and P. Pevzner, *Bioinformatics Algorithms: An Active Learning Approach*, vol. 1, 2015.
- [3] S. S. Skiena, *The algorithm design manual*. Springer, 2008.
- [4] T. Lecroq and C. Charras, *Handbook of Exact String Matching*. Rouen: Laboratoire d'Informatique de Rouen Université de Rouen, Faculté des Sciences et des Techniques 76821 Mont-Saint-Aignan Cedex, France, 2001.
- [5] V. SaiKrishna, P. A. Rasool, and D. N. Khare, "String Matching and its Application in Diversified Fields," *IJCSI Int. J. Comput. Sci. Issues*, vol. 9, no. 1, pp. 219–226, 2012.
- [6] G. A. Stephen, *String Searching Algorithms*, vol. 3. WORLD SCIENTIFIC, 1994.
- [7] N. Jiji and T. Mahalakshmi, "Survey of Exact String Matching Algorithm for Detecting Patterns in Protein Sequence," vol. 10, no. 8, pp. 2707–2720, 2017.
- [8] N. Singla and D. Garg, "String Matching Algorithms and their Applicability in various Applications," *Int. J. Soft Comput. Eng.*, no. 6, pp. 2231–2307, 2012.
- [9] M. Gou, "Algorithms for String matching," pp. 1–8, 2014.
- [10] R. S. Boyer and J. S. Moore, "A fast string searching algorithm," *Commun. ACM*, vol. 20, no. 10, pp. 762–772, Oct. 1977.
- [11] Ben Langmead, "Teaching Materials." [Online]. Available: <http://www.langmead-lab.org/teaching-materials/>.
- [12] G. Navarro and M. Raffinot, "Flexible Pattern Matching in Strings: Practical Online Search Algorithms for Texts and Biological Sequences," *Computer*, vol. 35, no. 9, 2002.
- [13] V. R. Knuth, D.E., Morris, J.H., & Pratt, J. H. Morris, Jr., and V. R. Pratt, "Fast Pattern Matching in Strings," *SIAM J. Comput.*, vol. 6, no. 2, pp. 323–350, 1977.